

UESC

UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC
CURSO SUPERIOR DE GRADUAÇÃO
EM CIÊNCIA DA COMPUTAÇÃO

BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT (ESCAPE-TIME): IMPLEMENTAÇÃO PARALELA EM MEMÓRIA DISTRIBUÍDA COM MPI

ILHÉUS - BA

30 de outubro de 2026

ALÍCIA OLIVEIRA ARAÚJO
ISABELA MADUREIRA ARGOLO

BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT (ESCAPE-TIME): IMPLEMENTAÇÃO PARALELA EM MEMÓRIA DISTRIBUÍDA COM MPI

Relatório apresentado como requisito parcial à disciplina de DEC107 - Processamento Paralelo, da Universidade Estadual de Santa Cruz, pelo curso de Ciência da Computação, turma de 2026.2.

Orientador (a): Prof. Dr. Esbel Tomás Valero Orellana.

ILHÉUS - BA

30 de outubro de 2026

SUMÁRIO

1. INTRODUÇÃO..... 4
2. METODOLOGIA..... 4
2.1. Implementação com MPI..... 4
2.2. Estratégias de Distribuição..... 5
2.2.1. Blocos Contíguos..... 5
2.2.2. Linhas Cíclicas..... 6
2.3. Saída e Cenários Utilizados..... 6
2.4. Ambiente dos Testes..... 7
2.5. Medição dos Tempos..... 7
2.6. Métricas Utilizadas..... 8
2.6.1. Speedup..... 8
2.6.2. Eficiência..... 8
2.6.3. Fator de Balanceamento de Carga..... 9
2.6.4. Escalabilidade Forte e Fraca..... 9
3. RESULTADOS..... 10
3.1. Teste de Corretude..... 10
3.2. Escalabilidade Forte..... 10
3.2.1. Vista Completa..... 10
3.2.2. Cavalos-Marinhos..... 11
3.3. Balanceamento de Carga..... 13
3.4. Custo de Comunicação e Escrita..... 14
3.5. Escalabilidade fraca..... 16
3.5.1. Vista Completa..... 16
3.5.2. Vista Completa..... 16
4. DISCUSSÃO..... 18
5. CONCLUSÃO..... 19
6. REFERÊNCIAS..... 19

1. INTRODUÇÃO

O conjunto de Mandelbrot pode ser representado pelo método *escape-time*, em que cada pixel corresponde a um ponto c do plano complexo. Partindo de $z_0 = 0$, aplica-se repetidamente a expressão $z_{n+1} = z_n^2 + c$ até que $|z_n|^2 > 4$ ou seja atingido o limite de iterações. A quantidade de iterações é utilizada na geração da imagem e reflete o custo computacional de cada ponto.

Como os pixels são calculados independentemente, a imagem pode ser dividida entre processos. No paralelismo em memória distribuída, cada processo possui seu próprio espaço de memória, e a comunicação ocorre pela troca de mensagens. Nesse contexto, o MPI (Message Passing Interface) oferece operações para coordenar esses processos e reunir os resultados.

A distribuição do trabalho pode ser estática, definida antes do cálculo, ou dinâmica, com tarefas atribuídas conforme os processos ficam disponíveis. Como o custo por pixel não é uniforme, uma divisão com quantidades semelhantes de pixels pode concentrar regiões mais custosas em poucos processos, aumentando a espera dos demais.

Nesta etapa do trabalho, o cálculo sequencial em C foi utilizado como base para uma implementação com MPI, comparando duas estratégias estáticas: blocos de linhas contíguas e distribuição cíclica de linhas. Foram analisados o balanceamento de carga, o speedup, a eficiência e a escalabilidade forte e fraca, além da comparação com OpenMP. Também foi considerado o custo de coleta da imagem com *MPI_Gatherv*, pois os ganhos do cálculo paralelo dependem tanto da distribuição do trabalho quanto da comunicação necessária para reunir os resultados.

2. METODOLOGIA

2.1. Implementação com MPI

A implementação segue o modelo SPMD, no qual todos os processos executam o mesmo programa, mas trabalham com parcelas diferentes dos dados.

O rank 0 recebe os parâmetros da execução e os envia aos demais processos com *MPI_Bcast*, onde, depois disso, cada processo identifica as linhas pelas quais será responsável e calcula suas contagens de iteração.

As principais funções utilizadas foram:

Função	Uso na Implementação
MPI_Init e *MPI_Finalize*	Inicialização e encerramento do *MPI*
MPI_Comm_rank	Identificação do processo
MPI_Comm_size	Obtenção da quantidade de processos
MPI_Bcast	Distribuição dos parâmetros
MPI_Barrier	Sincronização dos processos
MPI_Wtime	Medição dos tempos
MPI_Gatherv	Coleta das parcelas da imagem
MPI_Reduce	Obtenção das estatísticas dos tempos
MPI_Gather	Coleta dos tempos individuais na versão instrumentada para os testes

O MPI_Gatherv permite reunir parcelas com quantidades diferentes de elementos, sendo adequado quando a altura da imagem não é divisível pela quantidade de processos. Esse uso de contagens e deslocamentos também aparece no material sobre Scatter e Gather com o filtro gaussiano.

Não foi necessário utilizar *Scatter* para distribuir uma imagem inicial, pois os pontos são gerados a partir das coordenadas do plano complexo, e também não há troca de linhas de borda, já que o cálculo de um pixel não depende dos seus vizinhos.

2.2. Estratégias de Distribuição

2.2.1. Blocos Contíguos

Nessa estratégia, cada processo recebe um conjunto de linhas consecutivas. Quando existem linhas restantes na divisão, elas são distribuídas entre os primeiros processos.

A principal vantagem é a simplicidade da coleta, pois, como cada processo calcula uma faixa contínua, suas contagens podem ser colocadas diretamente na posição correspondente da matriz final.

Por outro lado, uma faixa pode conter mais pixels de alto custo do que outra. Nesse caso, alguns processos terminam antes e precisam esperar pelos demais.

2.2.2. Linhas Cíclicas

Na distribuição cíclica, o processo de rank r calcula as linhas da seguinte forma:

$$r,\ r + P,\ r + 2P,\ \ldots$$

onde P é a quantidade de processos.

Dessa forma, as linhas de uma mesma região são distribuídas entre processos diferentes. A intenção é evitar que apenas alguns processos recebam as partes mais caras da imagem. Depois do *Gatherv*, o rank 0 reorganiza as linhas, pois elas chegam agrupadas por processo.

As duas estratégias são estáticas, porque a atribuição das linhas é definida antes do cálculo. Não foi implementado mestre-trabalhador. O rank 0 também calcula sua parcela da imagem.

2.3. Saída e Cenários Utilizados

A saída principal é uma matriz de contagens em formato BIN, com inteiros de 32 bits em ordem *row-major*. Também é gerada uma imagem PPM para inspeção visual. A gravação é realizada pelo rank 0 e medida separadamente.

Foram utilizados os seguintes cenários:

Parâmetro	Vista Completa	Cavalos-Marinhos
Região	$Re \in [-2;\ 1]$, $lm \in [-1,5;\ 1,5]$	Centro em $(-0,743643887;\ 0,131825904)$
Largura Real do Zoom	–	$0,003$
Resolução Base	4096×4096	4096×4096
Máximo de Iterações	1000	5000
Precisão	*double*	*double*

A contagem segue a convenção da versão sequencial: registra o índice da iteração de escape, iniciado em zero, ou o limite de iterações quando o ponto não escapa.

2.4. Ambiente dos Testes

Os testes foram realizados em um computador com a seguinte configuração:

Item	Configuração
Processador	AMD Ryzen 7 5700U with Radeon Graphics
Núcleos Físicos	8
Processadores Lógicos	16
Sistema Operacional	Windows 11 Pro, versão 10.0.26200
Compilador	GCC 8.1.0, x86_64-posix-seh
MPI	Microsoft MPI, DLL versão 10.1.12498.52
OpenMP	`OMP_NUM_THREADS = P, OMP_DYNAMIC = FALSE, OMP_SCHEDULE = dynamic, 1`
Processos MPI / threads OpenMP	1, 2, 4 e 8

As versões foram compiladas com as opções **`-O3 -std=c11 -msse2 -mfpmath=sse -Wall -Wextra`**, utilizando também **`-fopenmp`** na implementação OpenMP.

Como todos os processos MPI foram executados no mesmo computador, os resultados representam a comunicação local entre processos e não o desempenho de um conjunto de máquinas conectado por rede.

2.5. Medição dos Tempos

Para cada configuração, foi realizada uma execução de aquecimento e três repetições para coleta, onde utilizamos a mediana das três repetições para comparar os resultados.

No MPI, uma barreira sincroniza os processos antes do início da medição e cada processo mede seu tempo de computação antes de esperar pelos demais. Depois, são medidos a espera na barreira, a coleta com *Gatherv* e a reordenação, quando utilizada.

Os intervalos são medidos com *MPI_Wtime*, que fornece o tempo decorrido no processo que realiza a chamada. O tempo paralelo considerado inclui cálculo das parcelas, espera pelos demais processos, coleta da imagem e reordenação das linhas cíclicas. A escrita, a validação, a impressão e a coleta das métricas ficam fora desse tempo.

Nas referências sequencial e OpenMP, a alocação da matriz está incluída no intervalo medido, e, no MPI, os buffers são alocados antes. Essa diferença deve ser considerada, principalmente quando os resultados são próximos.

2.6. Métricas Utilizadas

2.6.1. Speedup

O speedup foi calculado dividindo o tempo sequencial pelo tempo paralelo da mesma cena:

$$S(P) = \frac{\tilde{T}_{\text{seq}}}{\tilde{T}_{\text{par}}(P)}.$$

onde o símbolo T̃ representa a mediana das três repetições.

2.6.2. Eficiência

A eficiência foi calculada por:

$$E(P) = \frac{S(P)}{P}.$$

Para apresentar o resultado em porcentagem:

$$E_{\%}(P) = 100\, E(P).$$

No MPI, P representa processos. No OpenMP, representa threads.

2.6.3. Fator de Balanceamento de Carga

Para avaliar o desbalanceamento entre os processos, adotamos a seguinte definição de FBC:

$$FBC = \frac{T_{max} - T_{min}}{T_{max}}.$$

Nessa expressão, $T_{\max}$ e $T_{\min}$ são o maior e o menor tempo de computação entre os processos na mesma execução, sendo que comunicação e espera não entram nesse cálculo.

Um FBC próximo de zero indica melhor equilíbrio e valores próximos de 1 indicam maior desbalanceamento. O FBC foi calculado em cada repetição e, nas tabelas, apresentamos a mediana dos três resultados.

2.6.4. Escalabilidade Forte e Fraca

Na escalabilidade forte, mantivemos a resolução de 4096 × 4096 e variamos a quantidade de processos. Na escalabilidade fraca, aumentamos a resolução de acordo com:

$$N(P) \approx 4096\sqrt{P}.$$

Assim, a quantidade de pixels por processo permanece aproximadamente constante:

$$\frac{N(P)^2}{P} \approx 4096^2.$$

A eficiência fraca foi calculada separadamente para cada estratégia:

$$E_{\text{fraca}}(P) = \frac{\tilde{T}_{\text{MPI}}(1, 4096^2)}{\tilde{T}_{\text{MPI}}(P, N(P)^2)}.$$

Essa medida é diferente da eficiência forte, pois compara execuções com tamanhos de imagem diferentes.

3. RESULTADOS

3.1. Teste de Corretude

A corretude foi verificada comparando os hashes SHA256 das matrizes BIN completas.

As implementações MPI por blocos, MPI cíclica e OpenMP, com 8 recursos e resolução 4096 × 4096, apresentaram hashes iguais aos da versão sequencial nas duas cenas. As seis comparações registradas indicaram igualdade exata, sem necessidade de utilizar a tolerância de divergência permitida no enunciado.

Também foram realizados testes com 3 processos e resolução 65 × 65, verificando uma situação em que a altura não é divisível pela quantidade de processos.

3.2. Escalabilidade Forte

3.2.1. Vista Completa

O tempo sequencial mediano foi $7,552755$ s.

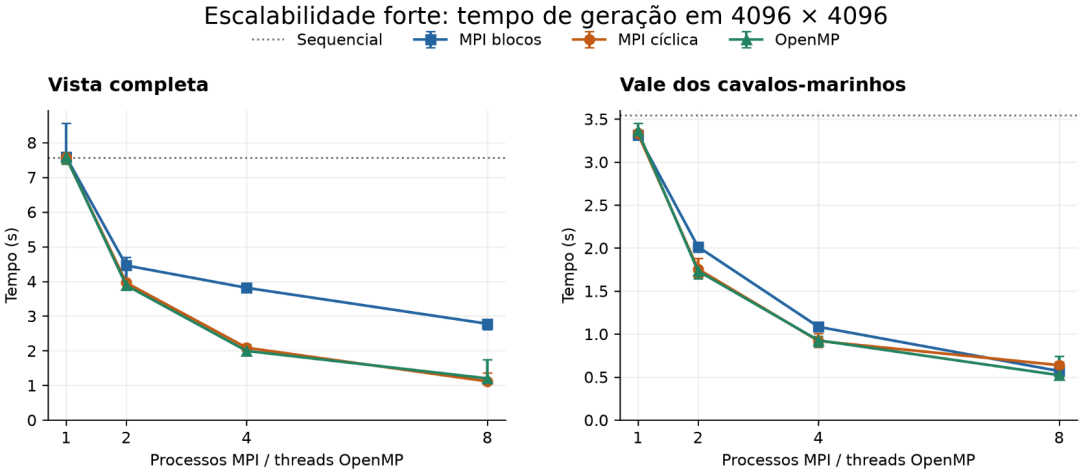
Implementação	P	Tempo Mediano (s)	Speedup	Eficiência (%)
MPI Blocos	1	$7,592$	$0,99$	$99,48$
MPI Blocos	2	$4,457$	$1,69$	$84,72$
MPI Blocos	4	$3,813$	$1,98$	$49,52$
MPI Blocos	8	$2,778$	$2,72$	$33,98$
MPI Cíclica	1	$7,571$	$1,00$	$99,75$
MPI Cíclica	2	$3,961$	$1,91$	$95,35$
MPI Cíclica	4	$2,086$	$3,62$	$90,52$
MPI Cíclica	8	$1,116$	$6,77$	$84,61$
OpenMP	1	$7,586$	$1,00$	$99,56$
OpenMP	2	$3,884$	$1,94$	$97,22$
OpenMP	4	$1,994$	$3,79$	$94,70$
OpenMP	8	$1,201$	$6,29$	$78,58$

3.2.2. Cavalos-Marinhos

O tempo sequencial mediano foi de $3,541781$ s.

Implementação	P	Tempo Mediano (s)	Speedup	Eficiência (%)
MPI Blocos	1	$3,315$	$1,07$	$106,85$
MPI Blocos	2	$2,015$	$1,76$	$87,90$
MPI Blocos	4	$1,084$	$3,27$	$81,68$
MPI Blocos	8	$0,570$	$6,22$	$77,69$
MPI Cíclica	1	$3,332$	$1,06$	$106,28$
MPI Cíclica	2	$1,756$	$2,02$	$100,86$
MPI Cíclica	4	$0,920$	$3,85$	$96,26$
MPI Cíclica	8	$0,638$	$5,55$	$69,36$
OpenMP	1	$3,361$	$1,05$	$105,39$
OpenMP	2	$1,732$	$2,04$	$102,22$
OpenMP	4	$0,927$	$3,82$	$95,55$
OpenMP	8	$0,522$	$6,79$	$84,83$

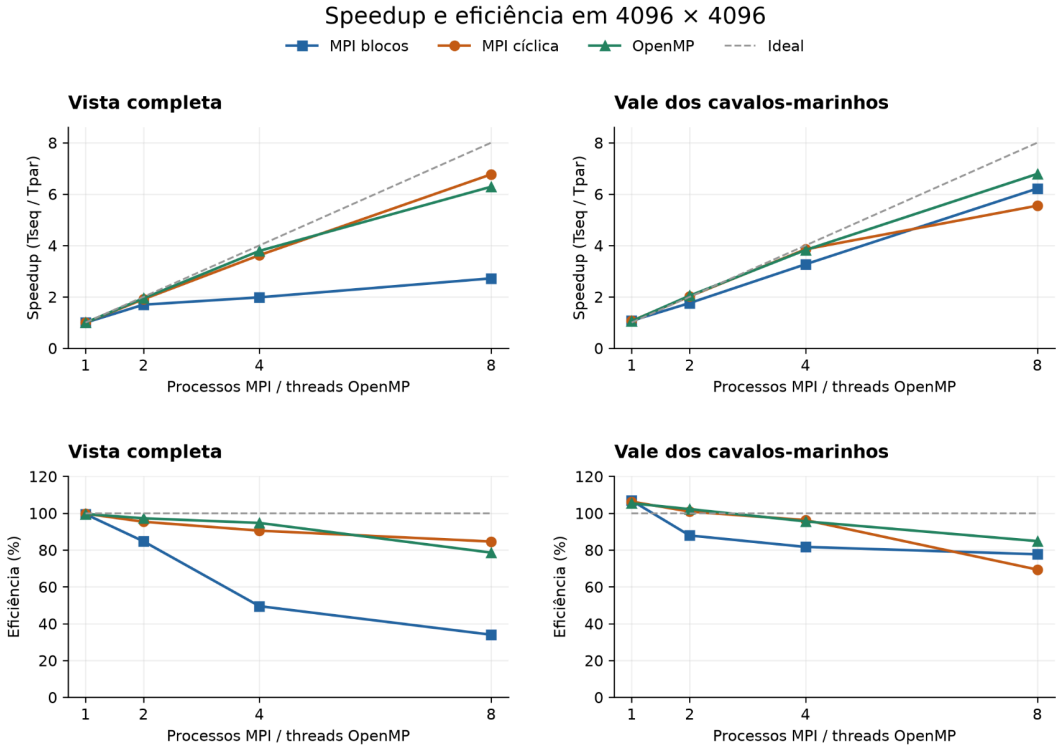
Figura 1 — Tempos de execução na escalabilidade forte



Fonte: Elaboração própria com os dados dos testes.

Obs.: Os pontos representam as medianas de três execuções, e as barras mostram o mínimo e o máximo das três repetições.

Figura 2 — Speedup e eficiência na escalabilidade forte



Fonte: Elaboração própria com os dados dos testes.

Obs.: As razões entre medianas têm a mesma referência sequencial por cena e valores maiores que 100% foram preservados e discutidos no texto.

Na vista completa, a distribuição cíclica apresentou a melhor mediana com 8 processos: $1,115849$ s. Em comparação com blocos, utilizando a mesma quantidade de processos, o tempo foi aproximadamente $59,83\%$ menor.

O OpenMP apresentou $1,201440$ s nessa configuração. Apesar da diferença entre as medianas, houve sobreposição entre os intervalos das repetições, por isso, não podemos afirmar que a cíclica será sempre mais rápida que OpenMP.

No zoom, o OpenMP com 8 threads apresentou a melhor mediana. Entre as versões MPI com 8 processos, blocos foi mais rápido que a cíclica, já com 2 e 4 processos, a cíclica apresentou menor tempo que blocos.

Algumas eficiências ficaram acima de 100%. Esses valores foram mantidos, porque resultam dos tempos medidos, porém, como já existem diferenças entre as versões com apenas um recurso, não interpretamos isso

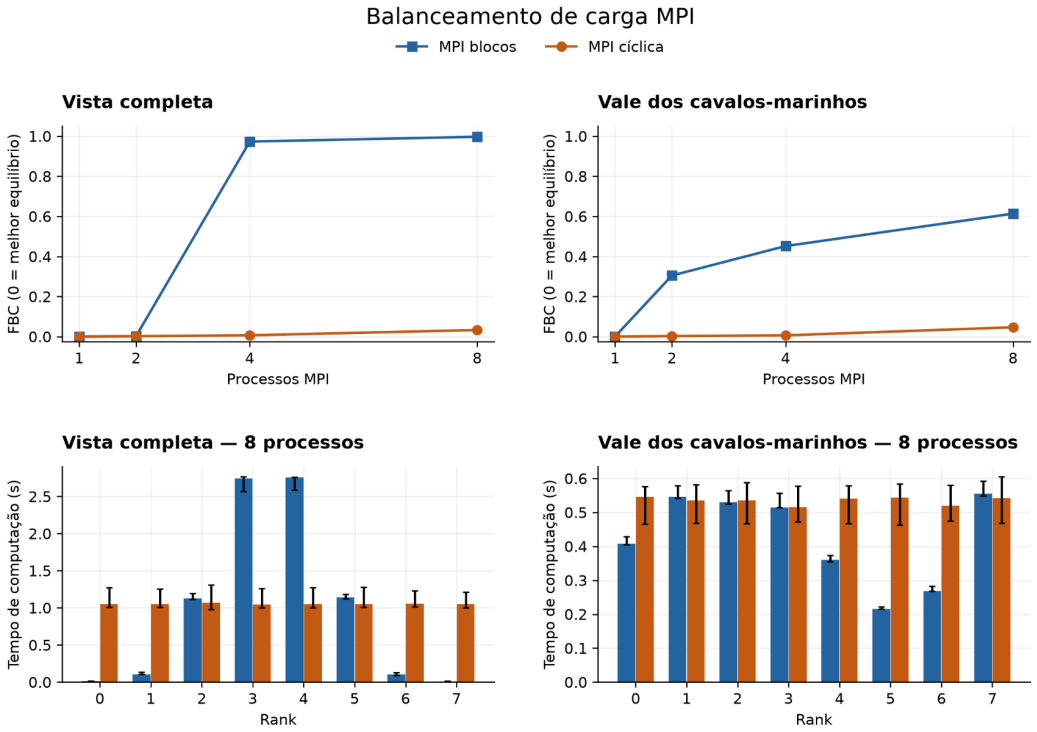
como uma comprovação de ganho superlinear. As diferenças na alocação, no código compilado e nas condições da máquina podem ter influenciado os resultados.

3.3. Balanceamento de Carga

Estratégia	P	FBC - Vista Completa	FBC - Cavalos-Marinhos
MPI Blocos	1	$0,0000$	$0,0000$
MPI Blocos	2	$0,0019$	$0,3045$
MPI Blocos	4	$0,9727$	$0,4515$
MPI Blocos	8	$0,9967$	$0,6133$
MPI Cíclica	1	$0,0000$	$0,0000$
MPI Cíclica	2	$0,0020$	$0,0026$
MPI Cíclica	4	$0,0065$	$0,0062$
MPI Cíclica	8	$0,0327$	$0,0464$

Os tempos medianos de computação por rank, com 8 processos, foram:

Figura 3 — Balanceamento de carga e tempos de computação por rank



Fonte: Elaboração própria com os dados dos testes

Na vista completa, os blocos apresentaram uma diferença grande entre os tempos dos processos. Os ranks responsáveis pelas faixas centrais ficaram com a maior parte do trabalho, enquanto outros terminaram rapidamente.

Na distribuição cíclica, os tempos ficaram mais próximos, o que mostra que alternar as linhas ajudou a distribuir o custo das regiões mais caras.

No zoom, a cíclica também apresentou menor FBC, mas, mesmo assim, ela não foi a mais rápida no teste forte com 8 processos. Esse resultado mostra que o balanceamento precisa ser analisado junto com os tempos de coleta e reordenação.

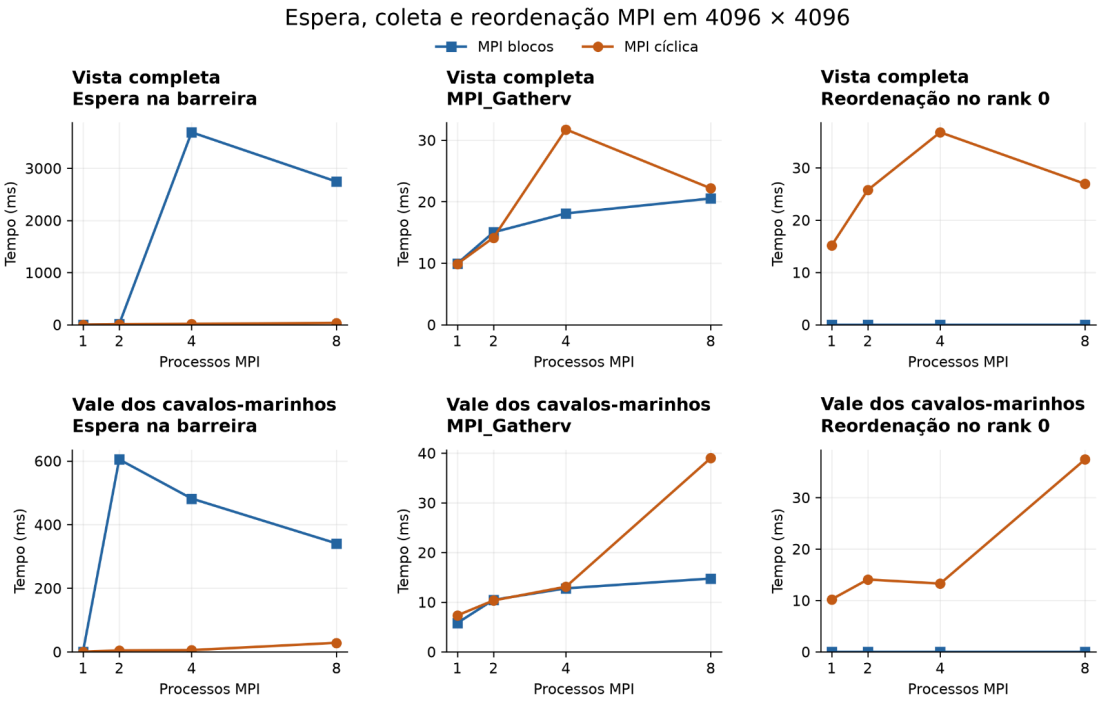
Os tempos por rank apresentados na Figura 3 são medianas de três repetições. Por isso, calcular o FBC diretamente a partir desses valores pode produzir um resultado diferente da mediana dos FBC calculados em cada execução.

3.4. Custo de Comunicação e Escrita

Com 8 processos, foram obtidas as seguintes medianas:

Cena	Estratégia	Espera Máxima (ms)	Gatherv Máximo (ms)	Reordenação (ms)
Vista Completa	MPI blocos	$2743,626$	$20,511$	$0,000$
Vista Completa	MPI cíclica	$33,184$	$22,159$	$26,914$
Cavalos-Marinhos	MPI blocos	$340,566$	$14,738$	$0,000$
Cavalos-Marinhos	MPI cíclica	$28,197$	$38,984$	$37,397$

Figura 4 — Tempos de espera, coleta e reordenação



Fonte: Elaboração própria com os dados dos testes

Na vista completa, o tempo de espera dos blocos foi muito maior que o tempo do Gatherv, o que indica que o desbalanceamento teve um impacto maior que o custo isolado da coleta.

Na cíclica, existe o custo adicional de reorganizar as linhas, já para a vista completa, esse custo foi compensado pela redução do tempo de cálculo e espera. No zoom com 8 processos, a coleta e a reordenação foram maiores, contribuindo para que a cíclica não apresentasse a menor mediana.

Os máximos de cada fase podem pertencer a ranks diferentes. Por isso, não devem ser somados para reconstruir o tempo total. Além disso, a espera na barreira não corresponde apenas à comunicação de dados.

A escrita foi medida separadamente:

Cena	Estratégia	P	Escrita BIN + PPM (s)
Vista Completa	MPI blocos	8	$1,108433$
Cavalos-marinhos	MPI blocos	8	$1,421431$
Vista Completa	MPI cíclica	8	$1,738648$
Cavalos-marinhos	MPI cíclica	8	$1,635082$

Esses tempos correspondem a uma única execução por cena e estratégia, durante a validação, não são medianas de três repetições. A medição inclui a gravação do BIN, a aplicação das cores e a gravação do PPM.

3.5. Escalabilidade fraca

Para manter aproximadamente constante a quantidade de pixels por processo, foram utilizadas as resoluções:

P	Resolução
1	4096×4096
2	5793×5793
4	8192×8192
8	11585×11585

3.5.1. Vista Completa

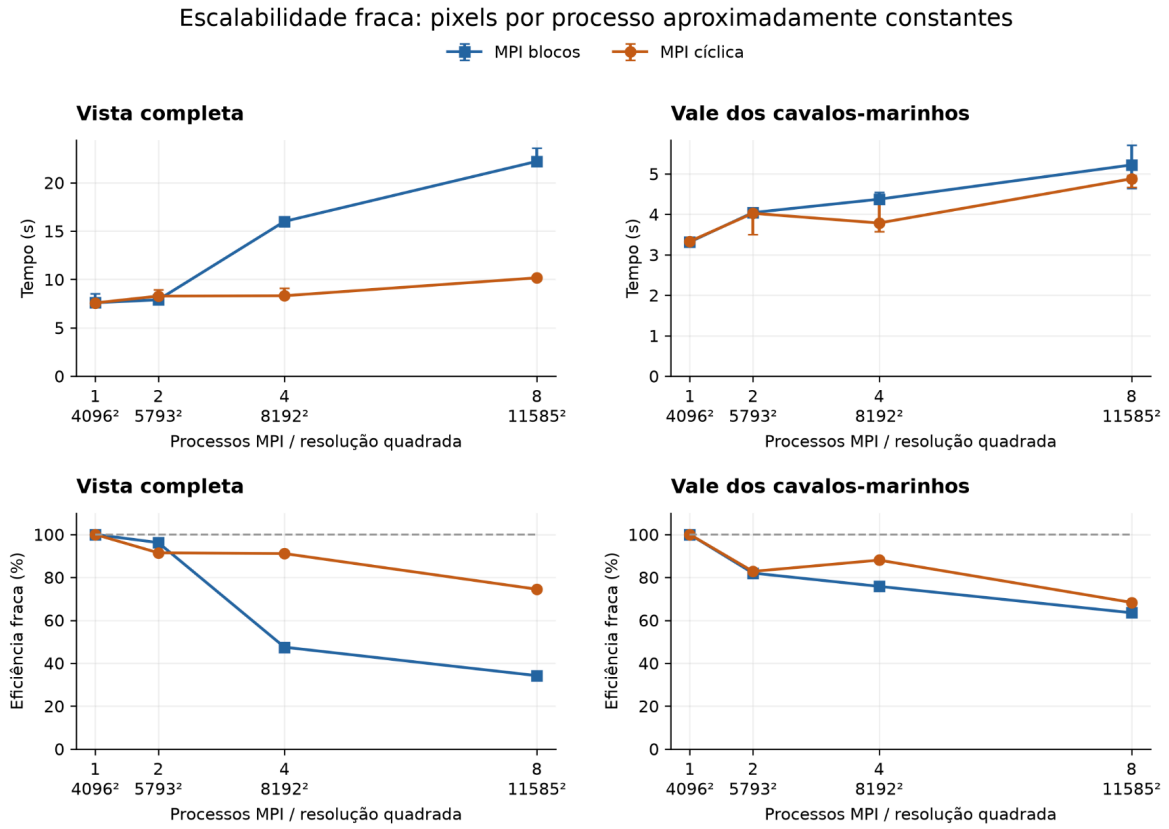
Estratégia	P	Tempo Mediano (s)	Eficiência fraca (%)
MPI blocos	1	$7,592$	$100,00$
MPI blocos	2	$7,892$	$96,19$
MPI blocos	4	$15,999$	$47,45$
MPI blocos	8	$22,208$	$34,19$
MPI cíclica	1	$7,571$	$100,00$
MPI cíclica	2	$8,279$	$91,45$
MPI cíclica	4	$8,312$	$91,09$
MPI cíclica	8	$10,167$	$74,47$

3.5.2. Vista Completa

Estratégia	P	Tempo Mediano (s)	Eficiência fraca (%)
MPI blocos	1	$3,315$	$100,00$
MPI blocos	2	$4,043$	$82,00$
MPI blocos	4	$4,373$	$75,80$

MPI blocos	8	$5,220$	$63,50$
MPI cíclica	1	$3,332$	$100,00$
MPI cíclica	2	$4,026$	$82,78$
MPI cíclica	4	$3,785$	$88,04$
MPI cíclica	8	$4,878$	$68,31$

Figura 5 — Tempo e eficiência na escalabilidade fraca



Fonte: Elaboração própria com os dados dos testes

Na vista completa, a diferença entre as estratégias aumentou nas maiores resoluções. Com 8 processos, a cíclica apresentou $10,166963$ s, enquanto blocos apresentou $22,207686$ s.

No zoom, a cíclica também apresentou menor mediana na maior resolução, embora a diferença tenha sido menor. Esse comportamento foi diferente do observado no teste forte com 8 processos.

Nenhuma estratégia manteve o tempo totalmente constante. O aumento da imagem também aumenta o volume de memória e de dados reunidos no rank 0. Na cíclica, aumenta ainda a quantidade de dados que precisa ser reorganizada.

4. DISCUSSÃO

A distribuição cíclica reduziu o desbalanceamento, principalmente na vista completa, ao distribuir as linhas das regiões mais custosas entre os processos. Os blocos contíguos apresentaram uma coleta mais simples, pois as parcelas recebidas já estavam na ordem da imagem.

No zoom, um FBC menor não garantiu o menor tempo total. Com 8 processos na escalabilidade forte, a distribuição por blocos apresentou menor mediana, mesmo com maior desbalanceamento. Na cíclica, os custos de coleta e reordenação foram maiores nessa configuração. Isso reforça a importância de analisar o equilíbrio da carga junto aos demais tempos da execução.

O OpenMP apresentou resultados próximos aos da cíclica em várias configurações. Como as threads compartilham a matriz, não é necessário reunir as parcelas da imagem por troca explícita de mensagens. Além disso, o escalonamento *dynamic,1* distribui novas linhas conforme as threads ficam disponíveis, enquanto as duas estratégias MPI possuem uma divisão definida antes do cálculo.

Na escalabilidade forte, os ganhos são limitados pelas partes sequenciais e pelos custos adicionais da execução paralela, comportamento relacionado à lei de Amdahl. Já a perspectiva de Gustafson considera o crescimento do problema com os recursos disponíveis, como ocorre nos testes de escalabilidade fraca. Entretanto, nessa implementação, o aumento do volume de dados e o desbalanceamento impediram a manutenção do tempo de execução.

Os resultados devem ser considerados dentro das condições dos testes. Todas as execuções ocorreram em uma única máquina, sem configuração explícita de afinidade e sem registro de temperatura, frequência da CPU ou atividade de outros programas. Também foram realizadas apenas três repetições por configuração. No OpenMP com 8 threads, por exemplo, os tempos da vista completa variaram de $1,139118$ s a $1,741280$ s, sem descarte de amostras.

Como continuidade, seria interessante aumentar a quantidade de repetições, controlar melhor as condições da máquina e avaliar uma estratégia mestre-trabalhador. Testes em máquinas conectadas por rede também permitiriam observar o custo de comunicação em outro ambiente.

5. CONCLUSÃO

A implementação com MPI permitiu comparar as distribuições por blocos contíguos e linhas cíclicas, mantendo os resultados das matrizes iguais aos da versão sequencial nas verificações realizadas.

A distribuição cíclica apresentou melhor balanceamento e menor tempo entre as versões MPI na vista completa. No zoom, a distribuição por blocos foi mais rápida em algumas configurações, mostrando que o balanceamento precisa ser considerado junto aos custos de coleta e reordenação. O OpenMP também apresentou desempenho competitivo no computador utilizado.

Os testes mostraram que dividir igualmente a quantidade de pixels não garante uma divisão equilibrada do trabalho. O desempenho depende do custo das regiões atribuídas a cada processo e da comunicação necessária para reunir a imagem final.

6. REFERÊNCIAS

ORELLANA, Esbel T. Valero. **DEC107 Processamento Paralelo: Aula 03 – Caracterização de desempenho em programas paralelos**. Ilhéus: Universidade Estadual de Santa Cruz, 2026.

AULA 10 — MPI: introdução e aplicações práticas. [S. l.: s. n.], [s. d.]. Slides da disciplina DEC107 — Processamento Paralelo, disponibilizados em aula.

AULA 11 — MPI: comunicações coletivas. [S. l.: s. n.], [s. d.]. Slides da disciplina DEC107 — Processamento Paralelo, disponibilizados em aula.

MPI — Scatter e Gather com o Filtro Gaussiano. [S. l.: s. n.], [s. d.]. Material didático da disciplina DEC107 — Processamento Paralelo, disponibilizado em aula.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, **Alícia Oliveira Araújo** e **Isabela Madureira Argolo**, regularmente matriculadas no curso de Ciência da Computação da Universidade Estadual de Santa Cruz, declaramos, para os devidos fins, que, na elaboração do trabalho intitulado **"Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação Paralela em Memória Distribuída com MPI"**, submetido como requisito para a disciplina **DEC107 — Processamento Paralelo**, utilizamos ferramentas de Inteligência Artificial generativa como recursos de apoio.

Ferramenta utilizada: ChatGPT.

Detalhamento do uso: a ferramenta foi utilizada como apoio nas seguintes etapas do projeto:

- auxílio na revisão do código na versão paralela com MPI;
- apoio em possíveis ajustes na distribuição do trabalho e na comunicação entre processos;
- auxílio na estruturação dos procedimentos de teste;
- geração de gráficos a partir dos dados coletados;
- revisão da organização, clareza e escrita do relatório.

Todo o código utilizado no projeto, os procedimentos de teste, os resultados obtidos e os textos produzidos com auxílio da ferramenta foram revisados e validados pelas autoras.

Termo de Responsabilidade: Declaramos estar cientes de que ferramentas de Inteligência Artificial podem apresentar erros, inconsistências ou informações incorretas. Dessa forma, assumimos integral responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido. Atestamos que o código-fonte, os resultados dos testes, as análises e os textos desenvolvidos com auxílio de IA foram revisados, testados e validados por nós, de forma a garantir a compreensão do trabalho realizado e a integridade acadêmica do material entregue.

Ilhéus – BA, 30 de outubro de 2026.

Alícia Oliveira Araújo
Matrícula: 202310348

Isabela Madureira Argolo
Matrícula: 202310353